



Semana 3 - Revisão

Problema A - Missing Numbers

Podemos criar uma array de frequências $freq$ onde nos indicará quantas vezes determinado número a_i foi visto na array arr . A quantidade de vezes que vimos o número a_i é dado por $freq[a_i]$.

Uma vez feito isso, ao ler os elementos da array brr descontamos de $freq$ os números vistos.

No final, podemos passar por $freq$ e os números faltando são aqueles onde $freq[a_i] \neq 0$.

Complexidade: $O(m)$

```
#include "stdlib.h"
#include "stdio.h"

int const MAX = 10001;

int main() {
    int n; scanf("%d",&n);

    int freq[MAX];
    for(int i = 0; i < MAX; i++) freq[i] = 0;

    for(int i = 0; i < n; i++) {
        int ai; scanf("%d",&ai);
        freq[ai]++;
    }

    int m; scanf("%d",&m);
    for(int i = 0; i < m; i++) {
        int bi; scanf("%d",&bi);
        freq[bi]--;
    }

    for(int i = 0; i < MAX; i++) {
        if(freq[i] != 0) printf("%d-",i);
    }
    printf("\n");
}
```

Problema B - Sherlock and Array

Utilizaremos uma técnica chamada "Soma de Prefixos (Prefix Sum)". Seja a uma array de n elementos, queremos calcular de maneira rápida a soma no intervalo

$$a[i] + a[i + 1] + \dots + a[j - 1] + a[j], \quad 1 \leq i, j \leq n.$$

Para isso, definimos a array $psum$ como a soma de prefixos de a , onde

$$psum[k] = \sum_{i=1}^k a[i] = psum[k - 1] + a[k].$$



Usando a definição, para calcular a soma do intervalo $[l, r]$ temos

$$\sum_{i=l}^r a[i] = \sum_{i=1}^r a[i] - \sum_{i=1}^{l-1} a[i] = psum[r] - psum[l-1].$$

Com isso, a resposta do problema se resume em encontrar um índice i , $1 \leq i \leq n$, tal que

$$psum[i-1] == psum[n] - psum[i+1].$$

Complexidade: $O(n)$

```
#include "stdio.h"
#include "stdlib.h"

int main(){
    int t; scanf("%d",&t);
    while(t--) {
        int n; scanf("%d",&n);

        int v[n];
        for(int i = 0; i < n; i++) {
            int ai; scanf("%d",&ai);
            v[i]=ai;
        }

        int psum[n+2];
        for(int i = 0; i < n+2; i++) psum[i] = 0;

        for(int i = 1; i <= n; i++) psum[i] = psum[i-1] + v[i-1];

        int ans = 0;
        for(int i = 1; i <= n && !ans; i++) ans = (psum[i-1] == psum[n] - psum[i]);

        if(ans) printf("YES\n");
        else printf("NO\n");
    }
}
```

Problema C - Binary Search - Advanced

Implementaremos a função `lower_bound`, nativa do C++, utilizando busca binária. Seja a uma array ordenada de n elementos, queremos encontrar o índice do primeiro elemento tal que seja maior ou igual a um número k . Para encontrá-lo, usaremos o seguinte algoritmo:

```
int lower_bound(int key, int n, int *v) {
    int ret = n;

    int l = 0, r = n-1;
    while(l <= r) {
        int mid = (l+r)/2;

        if(*(v+mid) >= key) { // acessando o valor dentro do ponteiro (v+mid)
            ret = mid;
            r = mid - 1; // refinamos a resposta
        } else {
            l = mid + 1;
        }
    }
}
```



}

return ret;

}

Com isso, resolvemos o problema por aplicação direta do algoritmo.

Complexidade: $O(n \log n)$

#include "stdio.h"

#include "stdlib.h"

```
int lb(int key, int n, int *v) {
    int ret = -1;
```

```
    int l = 0, r = n-1;
```

```
    while(l <= r) {
```

```
        int mid = (l+r)/2;
```

```
        if(*(v+mid) >= key) {
```

```
            ret = mid;
```

```
            r = mid-1;
```

```
        } else l = mid+1;
```

```
    }
```

```
    return ret;
```

}

```
int main() {
```

```
    int n; scanf("%d",&n);
```

```
    int v[n];
```

```
    for(int i = 0; i < n; i++) scanf("%d",&v[i]);
```

```
    int key; scanf("%d",&key);
```

```
    int idx_lb = lb(key,n,v), idx_ub = lb(key+1,n,v);
```

```
    if(idx_lb == -1 || v[idx_lb] != key) printf("-1-1-0\n");
```

```
    else printf("%d-%d-%d\n", idx_lb, idx_ub-1, idx_ub-idx_lb);
```

}

Problema D - Hash Tables: Ransom Note

Similar ao problema A, faremos uma array de frequências *hashes* com os hashes das palavras da revista.

A função de hash polinomial H será definida da seguinte forma:

$$H(s) = \sum_{i=0}^{|s|-1} (s[i] \times b^i) \pmod{p},$$

onde $s[i]$ é o valor do código ASCII do i -ésimo caractere de s , b a base da hash (no nosso caso adotaremos $b = 37$) e p um número primo relativamente grande para servir de identificação para as strings e evitar colisões (utilizaremos $p = 199999$).

Com isso definido, a solução segue da mesma ideia do problema A de descontar as hashes das palavras que queremos utilizar no bilhete de *hashes*, porém queremos saber se para toda hash h ($0 \leq h < p$) se $freq[h] \geq 0$.



Complexidade: $O(p)$

```
#include "stdio.h"
#include "stdlib.h"

int main() {
    int n,m; scanf("%d-%d", &n,&m);

    int const p = 37, mod = 199999;
    int hashes[mod], pot[5];
    for(int i = 0, aux = 1; i < 5; i++) {
        pot[i] = aux;
        aux *= p;
        aux %= mod;
    }

    for(int i = 0; i < mod; i++) hashes[i] = 0;

    for(int i = 0; i < n; i++) {
        char s[50]; scanf("%s",&s);

        int j = 0, h = 0;
        while(s[j] != '\0' && j < 5) {
            h += (s[j] * pot[j]);
            h %= mod;
            j++;
        }

        hashes[h]++;
    }

    for(int i = 0; i < m; i++) {
        char s[50]; scanf("%s",&s);

        int j = 0, h = 0;
        while(s[j] != '\0' && j < 5) {
            h += (s[j] * pot[j]);
            h %= mod;
            j++;
        }

        hashes[h]--;
    }

    int ans = 1;
    for(int i = 0; i < mod && ans; i++) ans = (hashes[i] >= 0);

    if(ans) printf("Yes\n");
    else printf("No\n");
}
```

Problema E - Ice Cream Parlor

Devido ao tamanho pequeno de n , é possível resolver este problema de maneira quadrática, isto é, testando cada par de custos do vetor *cost*.

Complexidade: $O(n^2)$



```
#include "stdio.h"
#include "stdlib.h"

int main() {
    int t; scanf("%d",&t);
    while(t) {
        int m,n; scanf("%d-%d", &m, &n);

        int v[n];
        for(int i = 0; i < n; i++) scanf("%d",&v[i]);

        int ans = 0;
        for(int i = 0; i < n && !ans; i++) {
            for(int j = i+1; j < n; j++) {
                if(v[i]+v[j]==m) {
                    printf("%d-%d\n", i+1,j+1);

                    ans = 1;
                    break;
                }
            }

            if(ans) break;
        }
    }
}
```